

```
"""
```

```
test_blob_local_collapse.py
```

```
=====
```

```
Test de l'analogie interne du trou noir dans le Blob.
```

```
Thèse à tester :
```

Le trou noir est un tunnel $3D \rightarrow 2D$ visible parce que la gravitation rencontre Ω_2 en espace restreint. Dans le Blob, $K4$ = analogue de matière. Les régions de forte densité $K4$ doivent donc présenter une dimension de Hausdorff locale d_H local effondrée par rapport au bulk.

```
Prédiction :
```

$\langle d_H \text{ local} \rangle(\text{dense}) < \langle d_H \text{ local} \rangle(\text{bulk})$,
idéalement avec $d_H \text{ local}(\text{dense}) \rightarrow 2$.

```
Méthode :
```

1. Équilibre MCMC via protocole O2v4 Phase A (validé).
2. $K4_count(v)$ = nombre de $K4$ contenant v (proxy local de masse).
3. $d_H \text{ local}(v)$ = pente $\log|V_r|$ vs $\log r$ sur la plage non-saturée autour de v (BFS, r petits).
4. Partition par $K4_count$: top 20% (dense) / mid 60% / bottom 20% (void).
5. Comparaison des distributions + test Welch.
6. Corrélation Spearman $K4_count \leftrightarrow d_H \text{ local}$ sur tous les vertices.

```
"""
```

```
import numpy as np
```

```
import math, time, json
```

```
from collections import deque
```

```
from scipy.spatial import KDTree
```

```
from scipy.stats import linregress, ttest_ind, spearmanr
```

```
CFG = {
```

```
    "target_degree": 8.0,  
    "proposal_radius_mult": 1.5,  
    "burn_in_factor": 100,  
    "prod_factor": 200,  
    "tau": 0.50,  
    "tau_scale_N": True,  
    "beta": 0.30,  
    "min_lcc_ratio": 0.90,
```

```
}
```

```
# ----- O2v4 functions (port validé) -----
```

```
def _rips_radius(N, cfg=CFG):
```

```

    return (cfg["target_degree"] / ((N-1) * (4/3) * math.pi)) ** (1/3)

def _E0(N, cfg=CFG):
    return int(N * cfg["target_degree"] / 2)

def _lcc_size(adj, N):
    visited = [False]*N; best = 0
    for s in range(N):
        if not visited[s]:
            sz = 0; q = deque([s]); visited[s] = True
            while q:
                v = q.popleft(); sz += 1
                for w in adj[v]:
                    if not visited[w]:
                        visited[w] = True; q.append(w)
            best = max(best, sz)
    return best

def _build_initial(N, seed, cfg=CFG):
    r = _rips_radius(N, cfg); r_p = cfg["proposal_radius_mult"] * r
    E0 = _E0(N, cfg)
    rng = np.random.default_rng(seed)
    pts = rng.uniform(0.0, 1.0, (N, 3))
    tree = KDTree(pts)
    adj = [set() for _ in range(N)]
    for i, j in tree.query_pairs(r):
        adj[i].add(j); adj[j].add(i)
    pool = list(tree.query_pairs(r_p))
    rng2 = np.random.default_rng(seed + 50_000)
    K_curr = sum(len(adj[i]) for i in range(N)) // 2
    max_att = 10 * abs(E0 - K_curr) + 5000
    attempts = 0
    while K_curr != E0 and attempts < max_att:
        attempts += 1
        if K_curr < E0:
            e = pool[rng2.integers(len(pool))]
            u, v = e
            if v not in adj[u]:
                adj[u].add(v); adj[v].add(u); K_curr += 1
        else:
            edges = [(i,j) for i in range(N) for j in adj[i] if j>i]
            if not edges: break
            e = edges[rng2.integers(len(edges))]
            u, v = e
            adj[u].discard(v); adj[v].discard(u)
            if _lcc_size(adj, N) == N: K_curr -= 1
            else: adj[u].add(v); adj[v].add(u)

```

```

    return pts, adj, pool, E0

def _count_triangles(adj, N):
    n = 0
    for i in range(N):
        for j in adj[i]:
            if j > i:
                n += len([k for k in adj[i] & adj[j] if k > j])
    return n

def compute_S(adj, n_tri, N, cfg=CFG):
    degs = np.array([len(adj[i]) for i in range(N)], float)
    return math.log(1 + n_tri) - cfg["beta"] * degs.std() / (degs.mean() + 1e-8)

def _fast_still_connected(adj, u, v, N):
    visited = {u}; q = deque([u])
    while q:
        node = q.popleft()
        for w in adj[node]:
            if w in visited: continue
            if node == u and w == v: continue
            if node == v and w == u: continue
            visited.add(w); q.append(w)
            if w == v: return True
    return False

def run_mcmc(pts, adj_init, pool, N, E0, seed, cfg=CFG):
    tau = cfg["tau"] / N
    rng = np.random.default_rng(seed + 200_000)
    adj = [set(a) for a in adj_init]
    edge_list = [(i,j) for i in range(N) for j in adj[i] if j>i]
    n_tri = _count_triangles(adj, N)
    S_curr = compute_S(adj, n_tri, N, cfg)
    n_steps = (cfg["burn_in_factor"] + cfg["prod_factor"]) * N
    u_arr = rng.random(size=n_steps)
    pool_arr = np.array(pool, dtype=np.int32)
    n_pool = len(pool_arr)
    for step in range(n_steps):
        if not edge_list: break
        idx_out = int(rng.integers(len(edge_list)))
        u, v = edge_list[idx_out]
        e_in = None
        for _ in range(30):
            idx_in = int(rng.integers(n_pool))
            a, b = int(pool_arr[idx_in,0]), int(pool_arr[idx_in,1])
            if b not in adj[a]:
                e_in = (a, b); break

```

```

        if e_in is None: continue
        a, b = e_in
        if len(adj[u]) <= 2 or len(adj[v]) <= 2:
            if not _fast_still_connected(adj, u, v, N): continue
        tri_lost = len(adj[u] & adj[v])
        adj[u].discard(v); adj[v].discard(u)
        adj[a].add(b); adj[b].add(a)
        tri_gained = len(adj[a] & adj[b])
        n_tri_new = n_tri - tri_lost + tri_gained
        S_new = compute_S(adj, n_tri_new, N, cfg)
        dS = S_new - S_curr
        if dS >= 0 or u_arr[step] < math.exp(dS / tau):
            n_tri = n_tri_new; S_curr = S_new
            edge_list[idx_out] = (a, b)
        else:
            adj[u].add(v); adj[v].add(u)
            adj[a].discard(b); adj[b].discard(a)
    return adj, n_tri

# ----- Observables locaux -----
def compute_k4_per_vertex(adj, N):
    """Compte par vertex le nombre de K4 (4-cliques) le contenant."""
    k4_count = np.zeros(N, dtype=int)
    for i in range(N):
        for j in adj[i]:
            if j <= i: continue
            common_ij = adj[i] & adj[j]
            for k in common_ij:
                if k <= j: continue
                common_ijk = common_ij & adj[k]
                for l in common_ijk:
                    if l <= k: continue
                    k4_count[i] += 1
                    k4_count[j] += 1
                    k4_count[k] += 1
                    k4_count[l] += 1
    return k4_count

def compute_dH_local(adj, source, N, max_radius=8, saturation=0.6):
    """d_H local autour de source : pente log|V_r| vs log r sur la plage non-satu
    dist = -np.ones(N, dtype=int)
    dist[source] = 0
    frontier = deque([source])
    while frontier:
        v = frontier.popleft()
        if dist[v] >= max_radius: continue
        for u in adj[v]:

```

```

        if dist[u] == -1:
            dist[u] = dist[v] + 1
            frontier.append(u)
pairs = []
for r in range(1, max_radius + 1):
    vol = int(((dist >= 0) & (dist <= r)).sum())
    if 1 < vol < saturation * N:
        pairs.append((r, vol))
if len(pairs) < 3:
    # Fallback : prendre tous les r où vol > 1, jusqu'à pré-saturation
    pairs = [(r, int(((dist >= 0) & (dist <= r)).sum())) for r in range(1, ma
    pairs = [(r, v) for r, v in pairs if v > 1 and v < N]
    if len(pairs) < 3: return None
rs = np.array([r for r, _ in pairs], dtype=float)
vs = np.array([v for _, v in pairs], dtype=float)
slope, _ = np.polyfit(np.log(rs), np.log(vs), 1)
return float(slope)

# ----- Main -----
if __name__ == "__main__":
    N = 500
    seeds = [0, 1, 2, 3, 4]
    print(f"=== Test collapse dimensionnel local - N={N}, {len(seeds)} seeds ===")
    print(f"Hypothèse : d_H_local(dense K4) < d_H_local(bulk)\n")

    all_records = [] # tuples (seed, vertex, k4_count, dH_local)
    seed_summary = []

    t_total = time.time()
    for seed in seeds:
        t0 = time.time()
        pts, adj_init, pool, E0 = _build_initial(N, seed, CFG)
        K_init = sum(len(adj_init[i]) for i in range(N)) // 2
        if _lcc_size(adj_init, N) / N < CFG["min_lcc_ratio"] or K_init != E0:
            print(f" seed {seed}: rejected initial")
            continue
        adj, n_tri = run_mcmc(pts, adj_init, pool, N, E0, seed, CFG)
        t_mcmc = time.time() - t0

        t1 = time.time()
        k4_count = compute_k4_per_vertex(adj, N)
        t_k4 = time.time() - t1

        t2 = time.time()
        dH_locals = []
        for v in range(N):
            dh = compute_dH_local(adj, v, N)

```

```

        dH_locals.append(dh)
        all_records.append((seed, v, int(k4_count[v]), dh))
t_dh = time.time() - t2

dH_arr = np.array([x for x in dH_locals if x is not None])
print(f"  seed {seed}: n_tri={n_tri}, n_K4_inc_total={int(k4_count.sum()/
        f"k4_count: max={k4_count.max()}, mean={k4_count.mean():.2f}  "
        f"d_H_local mean={dH_arr.mean():.3f}  "
        f"[mcmc {t_mcmc:.1f}s, k4 {t_k4:.1f}s, dH {t_dh:.1f}s]")
seed_summary.append({'seed': seed, 'n_tri': n_tri,
                    'n_K4': int(k4_count.sum()/4),
                    'k4_max': int(k4_count.max()),
                    'dH_local_mean': float(dH_arr.mean())})

# ----- Analyse globale -----
records = [r for r in all_records if r[3] is not None]
k4_arr = np.array([r[2] for r in records])
dH_arr = np.array([r[3] for r in records])
print(f"\nTotal vertices analysés : {len(records)} (sur {N * len(seeds)})")

# Partition par K4_count
q20 = np.quantile(k4_arr, 0.20)
q80 = np.quantile(k4_arr, 0.80)
void_mask = k4_arr <= q20
bulk_mask = (k4_arr > q20) & (k4_arr < q80)
dense_mask = k4_arr >= q80

print(f"\nSeuils K4_count : q20={q20:.0f}, q80={q80:.0f}")
print(f"  Void  (K4_count ≤ {q20:.0f}) : n={void_mask.sum()},  {K4}={k4_arr[v
print(f"  Bulk  ({q20:.0f} < K4_count < {q80:.0f}) : n={bulk_mask.sum()},  {K
print(f"  Dense (K4_count ≥ {q80:.0f}) : n={dense_mask.sum()},  {K4}={k4_arr[

print(f"\n--- d_H_local par catégorie ---")
for label, m in [("Void", void_mask), ("Bulk", bulk_mask), ("Dense", dense_ma
    d = dH_arr[m]
    if len(d) > 0:
        print(f"  {label:<6} : n={len(d):>4}  {d_H_local} = {d.mean():.3f} ±
            f"median = {np.median(d):.3f}  range=[{d.min():.2f}, {d.max():.

# Test statistique : Dense vs Bulk
print(f"\n--- Test statistique (Welch t-test) ---")
if dense_mask.sum() >= 5 and bulk_mask.sum() >= 5:
    t, p = ttest_ind(dH_arr[dense_mask], dH_arr[bulk_mask], equal_var=False)
    diff = dH_arr[dense_mask].mean() - dH_arr[bulk_mask].mean()
    print(f"  Dense vs Bulk : Δ{d_H_local} = {diff:+.3f}  t = {t:+.2f}  p = {
    verdict_db = "Dense < Bulk (cohérent avec collapse)" if diff < 0 and p <
        "Dense > Bulk (anti-prédiction)" if diff > 0 and p < 0.01 el

```

```

print(f" Verdict : {verdict_db}")

if dense_mask.sum() >= 5 and void_mask.sum() >= 5:
    t, p = ttest_ind(dH_arr[dense_mask], dH_arr[void_mask], equal_var=False)
    diff = dH_arr[dense_mask].mean() - dH_arr[void_mask].mean()
    print(f" Dense vs Void :  $\Delta(d_H_{local}) = \{diff:+.3f\}$   t = {t:+.2f}  p = {

# Corrélation Spearman
rho, p_rho = spearmanr(k4_arr, dH_arr)
print(f"\n Corrélation Spearman K4_count ↔ d_H_local :  $\rho = \{rho:+.3f\}$   p = {
if rho < -0.2 and p_rho < 0.01:
    print(f" ⇒ Corrélation négative significative : plus K4_count est élevé
    print(f" ⇒ Compatible avec la thèse 'régions denses = collapse dimensio
elif rho > 0.2 and p_rho < 0.01:
    print(f" ⇒ Corrélation POSITIVE significative : opposé à la prédiction.
else:
    print(f" ⇒ Pas de corrélation significative.")

# Save
out = {
    'seeds': seeds, 'N': N,
    'seed_summary': seed_summary,
    'partition_thresholds': {'q20': float(q20), 'q80': float(q80)},
    'stats': {
        'void': {'n': int(void_mask.sum()), 'dH_mean': float(dH_arr[void_ma
        'bulk': {'n': int(bulk_mask.sum()), 'dH_mean': float(dH_arr[bulk_ma
        'dense': {'n': int(dense_mask.sum()), 'dH_mean': float(dH_arr[dense_m
    },
    'spearman': {'rho': float(rho), 'p': float(p_rho)},
    'wall_time_s': float(time.time() - t_total),
}
with open('/home/claude/local_collapse_results.json', 'w') as f:
    json.dump(out, f, indent=2)
print(f"\nTemps total : {time.time() - t_total:.1f}s")
print(f"Sauvegarde : /home/claude/local_collapse_results.json")

```